[Project #2] Newsfeed with GraphQL

Feedback Deadline: **May 21, 2025 @ 11:59 PM CT**

Cutoff Date: **May 28, 2025 @ 11:59 PM CT (No feedback will be given)**

GitHub Classroom: https://classroom.github.com/a/0fJ2wJF2 (Please don't remove the feedback PR)

TL;DR: Create a newsfeed application using Next.js with GraphQL backend in Django using Ariadne, leveraging AI tools to assist your development process.

Overview

In this project, you'll be creating a barebones version of a newsfeed using Next.js and Django (Springboot if you're looking for a challenge).

Parts

The FE

Since you're an expert now in FE, now's the chance to enhance your productivity by prompt engineering + vibe-coding a FE and evaluating the output.

This will give you an opportunity to practice/develop your prompt engineering + vibe coding skills.

- What's the difference? Prompt eng == more specific than vibe-coding. They're both complementary approaches to getting what you want out of your tool.
- Prompt eng when you have a very specific technical requirement
- Vibe code when you want to defer to the AI to make stylistic/creative choices

General flow

1. Vibe code an foundational app
2. Review the resultant code and refine with prompt eng

How to vibe-code

There are a few ways to do this: you can either

a. have the tool give you the FE app all at once (I personally don't like this approach since it makes testing a pain...)
b. iterate based on features (you're in charge of wiring everything up yourself and ensuring that things work together/are consistent.

Either way, you'll need to evaluate that things work and that you're ensuring best practice with your FE knowledge – performance considerations, security, etc.

How to prompt eng (a ultra simplified/basic guide)

1. Ground your model – this will help guide the model and reduce hallucinations
    a. Ex: "You're an expert Next.js engineer who considers performance tradeoffs..."
2. State what you're looking for – be specific.
    a. Ex: Refactor this code to use components thinking by using the atomic design principles and composability.
3. Few shot examples:
    a. Add a few examples with:
        i. Input code – perhaps a messy monolithic component
        ii. Desired result – state what you'd separate out into components

Requirements

- **Build your own components instead of using UI libraries like Material UI**
- Create a responsive newsfeed interface that displays posts with:
    - Title, content, author, timestamp
    - [stretch] Like/comment functionality
    - Post creation, editing, and deletion features
- [stretch] Implement filtering/sorting options (newest, most liked, search, etc.)

The BE

You could also try to vibe code the BE (but do so at your own risk especially if you're brand new to Django/GraphQL then it might make sense to use AI tooling for debugging instead).

Requirements

- Implement GraphQL API using Django with Ariadne or Strawberry ~~Ariadne (not Strawberry or Graphene)~~
- Create appropriate Django models for posts, users, comments, etc.
- Define GraphQL schema with:

    - Queries: fetch all posts, single post, filtered posts
    - Mutations: create, update, delete posts
- Ensure proper error handling and validation

Frontend-Backend Integration

- Connect your React frontend to your GraphQL API (as shown in class)
- Ensure changes persist (refreshing should maintain state)
    - By default, your Django model does this (assuming you have the same setup from class)

Mini Design Doc

General template:

i. Overview/Background ii. **Technical** problem statement iii. Design Approach

- Walk me through your thought process:
- How you used AI throughout development
- Examples of effective/ineffective prompts
- Your component architecture decisions
- How you designed your GraphQL schema
- Challenges encountered and solutions implemented
- Anything else? (AI tooling effectiveness, what you learned, etc.)

For Self-Evaluation

1. Code quality
    - Clean, readable code with appropriate comments
    - No extraneous console logs
2. Components
    - Build your own components instead of using libraries like Material UI
    - Appropriate component separation and reuse
    - Effective props and state management
3. GraphQL Implementation
    - Well-structured schema design
    - Efficient queries and mutations
    - Proper integration with frontend
4. AI Collaboration
    - Strategic use of AI assistance

    - Evidence of learning and iteration from AI suggestions
    - Ability to explain all code regardless of AI generation
5. User Experience
    - Intuitive interface with smooth interactions
    - Responsive design that works on various screen sizes
    - Proper loading and error states
    - Semantic HTML (for accessibility – you can have AI do the work here too)

Deliverable (place all deliverables in your GitHub classroom repo)

1. Your code -- include both frontend and backend code
    - Be sure to add a .gitignore file to exclude node_modules and any virtual environments
    - Include setup instructions in your README
2. AI Collaboration Documentation
    - Include examples of your interactions with AI tools
    - Highlight what you learned from these interactions
    - Reflect on effective prompting strategies
3. Link to your mini design doc in the README -- be sure that your teaching team has viewer access

FAQs

Want more challenges?

- Consider implementing more advanced features like:
    - Real-time updates with subscriptions
    - Authentication/authorization
    - Infinite scrolling or pagination
    - More complex filtering options

How much should I rely on AI tools?

- There's no set percentage, but you should understand every line of code in your project
- Use AI as a collaborator to learn from, not just a solution generator

- The quality of your prompts and how you refine AI suggestions matters more than quantity

Setting Up Your Project

Frontend Setup

Update Jun 5, 2025 → incorrect setup instructions (should use Next.js as mentioned above)

~~# Using Create React App~~
~~npx create-react-app newsfeed-frontend~~
~~cd newsfeed-frontend~~
~~npm install @apollo/client graphql~~

Or using Next.js
```
npx create-next-app@latest newsfeed-frontend
cd newsfeed-frontend
npm install @apollo/client graphql
```

Backend Setup

We'll be using Poetry for Python dependency management. Poetry provides several advantages over traditional pip/venv:

- Automatically manages virtual environments
- Single file (pyproject.toml) for both dependencies and project metadata
- Lockfile system ensures consistent installations across environments
- Better dependency resolution than pip

```
# Install Poetry if you don't have it already
# curl -sSL https://install.python-poetry.org | python3 -

# Create a new project
mkdir newsfeed_backend
cd newsfeed_backend

# Initialize Poetry project
poetry init --no-interaction --name newsfeed_backend --description
"Newsfeed GraphQL Backend"
```

```
# Add dependencies
poetry add django ariadne django-cors-headers

# Create Django project within Poetry environment
poetry run django-admin startproject newsfeed_backend .
poetry run python manage.py startapp api
```

!Important: Remember to set up CORS headers to allow your frontend to communicate with your backend during development.